

4.7 - JWT: estrutura, validação e armadilhas

O token que todos usam e poucos validam direito

As três partes de um JWT

Header, payload e signature

- **Header descreve o algoritmo e o tipo do token**
 - Define como a assinatura deve ser verificada
- **Payload carrega os claims sobre a identidade**
 - Claims padrão: iss, sub, aud, exp, iat
- **Signature garante que nada foi alterado**
 - Assinada com chave simétrica ou par assimétrico
- **Base64url não é criptografia, o payload é legível**
 - Nunca coloque segredo no payload de um JWT

Anatomia de um JWT

JSON

```
// header
{ 'alg': 'RS256', 'typ': 'JWT' }

// payload (claims)
{
  'iss': 'https://idp.lunalog.com',
  'sub': 'user-8842',
  'aud': 'lunalog-api',
  'exp': 1716800000,
  'roles': ['motorista']
}

// token transmitido
base64url(header).base64url(payload).signature
```

Validar um JWT corretamente

- **Verificar a assinatura antes de confiar em qualquer claim**
 - Usar a chave pública correta do issuer
- **Checar exp para rejeitar token expirado**
 - Token sem expiração é credencial permanente
- **Validar aud para impedir reuso em outro serviço**
 - Validar iss para confiar só no emissor esperado
- **Rejeitar alg none, a armadilha clássica do JWT**

As armadilhas que viram vulnerabilidade

- **alg none desliga a verificação de assinatura**
- **Aceitar o algoritmo do header confia no atacante**

Sempre fixe o algoritmo esperado no servidor. Deixar o token escolher o próprio algoritmo de validação é entregar a chave da fechadura.

- Token longo sem revogação é dano que não para

Tokens que nunca expiravam

- **Auditoria de segurança examina os JWT da plataforma**
 - Tokens emitidos sem campo de expiração

LUNALOG

Uma auditoria de segurança contratada pela LunaLog encontrou tokens JWT emitidos pela plataforma sem o campo de expiração. Tokens criados quatorze meses antes continuavam tecnicamente válidos. Na prática, um motorista demitido de uma transportadora havia um ano ainda poderia acessar o sistema com o token antigo que tinha guardado. Ninguém havia decidido emitir tokens eternos, simplesmente esqueceram de incluir o exp. Um checklist de validação de JWT teria capturado essa falha antes do primeiro token entrar em produção. Ana transformou esse checklist em requisito de revisão de código.

Quando JWT é ou não a ferramenta

JWT faz sentido

- Autenticação stateless entre serviços
- Claims que mudam pouco durante a sessão
- Sistemas distribuídos sem sessão central
- Tokens de curta duração e escopo claro

JWT não é a melhor escolha

- Necessidade de revogação imediata
- Sessão que precisa ser invalidada na hora
- Dados sensíveis que não podem ser legíveis
- Claims que mudam a cada requisição



Seu token está bem estruturado, validado e com expiração. Mesmo assim, um único fator de autenticação ainda é frágil demais para proteger dados sensíveis. E há uma pergunta que muita arquitetura ignora: como os próprios serviços se autenticam entre si, sem depender de uma credencial humana?

MFA, SSO e autenticação entre serviços